

Milestone 1 — Python OOPS Solutions

Complete solutions for all 3 OOPS problem sets from the supplied sample-question PDF. Each problem includes all functions.

Question 1 — Supply Chain Inventory Tracker

Complete Code

```
class SupplyChainInventory:

    def __init__(self):
        self.inv = {}

    def add_product(self, product_id: str, quantity: int) -> dict:
        if product_id in self.inv:
            self.inv[product_id] += quantity
        else:
            self.inv[product_id] = quantity
        return self.inv

    def fulfill_order(self, product_id: str, quantity: int) -> dict:
        if product_id not in self.inv:
            raise ValueError("Product not found")
        if self.inv[product_id] < quantity:
            raise ValueError("Insufficient Stock")
        self.inv[product_id] -= quantity
        return self.inv

    def restock_return(self, product_id: str, quantity: int) -> dict:
        if product_id in self.inv:
            self.inv[product_id] += quantity
        else:
            self.inv[product_id] = quantity
        return self.inv

    def list_available_products(self) -> list:
        return [
            product_id
            for product_id, quantity in self.inv.items()
            if quantity > 0
        ]

# Example
obj = SupplyChainInventory()
print(obj.add_product("ProductB", 10))
print(obj.add_product("ProductB", 5))
print(obj.fulfill_order("ProductB", 6))
print(obj.restock_return("ProductB", 10))
print(obj.list_available_products())
```

Easy Explanation

- `add_product`: existing product gets additional stock; otherwise a new entry is created.
- `fulfill_order`: checks stock and subtracts the requested quantity. Insufficient stock raises `ValueError`.
- `restock_return`: adds returned quantity, or creates the product if it does not exist.
- `list_available_products`: returns product IDs whose stock is greater than zero.

Question 2 — Event Registration Tracker

Complete Code

```
class EventRegistrationTracker:

    def __init__(self):
        self.events = {}

    def register_participant(self, event_id: str, count: int) -> dict:
        if event_id in self.events:
            self.events[event_id] += count
        else:
            self.events[event_id] = count
        return self.events

    def cancel_participant(self, event_id: str, count: int) -> dict:
        if event_id not in self.events:
            raise ValueError(
                "Cannot cancel more participants than registered"
            )
        if self.events[event_id] < count:
            raise ValueError(
                "Cannot cancel more participants than registered"
            )
        self.events[event_id] -= count
        return self.events

    def reschedule_event(self, old_event_id: str, new_event_id: str) -> dict:
        if old_event_id not in self.events:
            return self.events

        count = self.events[old_event_id]

        if new_event_id in self.events:
            self.events[new_event_id] += count
        else:
            self.events[new_event_id] = count

        del self.events[old_event_id]
        return self.events

    def get_active_events(self) -> list:
        return [
            event_id
            for event_id, count in self.events.items()
            if count > 0
        ]

# Example
obj = EventRegistrationTracker()
print(obj.register_participant("Gaming", 10))
print(obj.register_participant("Gaming", 10))
print(obj.cancel_participant("Gaming", 5))
print(obj.register_participant("Trekking", 25))
print(obj.reschedule_event("Gaming", "Trekking"))
print(obj.get_active_events())
```

Easy Explanation

- `register_participant`: increases an existing event's count or creates a new event.
- `cancel_participant`: raises `ValueError` if the event is missing or there are not enough registered participants.
- `reschedule_event`: moves the old event's participant count to the new event and deletes the old event.
- `get_active_events`: returns event IDs with participant count greater than zero.

Question 3 — Smart City Traffic Control System

Complete Code

```
class TrafficControlSystem:

    def __init__(self):
        self.traffic_data = {}

    def add_intersection(self, intersection: str, vehicle_count: int) -> dict:
        self.traffic_data[intersection] = vehicle_count
        return self.traffic_data

    def update_vehicle_count(self, intersection: str, new_count: int) -> dict:
        if intersection not in self.traffic_data:
            return "Error: Intersection not found"

        self.traffic_data[intersection] = new_count
        return self.traffic_data

    def get_congested_intersections(self, congestion_threshold: int) -> dict:
        result = {}

        for intersection, count in self.traffic_data.items():
            if count > congestion_threshold:
                result[intersection] = count

        return result

    def adjust_traffic_signals(self) -> dict:
        result = {}

        for intersection, count in self.traffic_data.items():
            if count > 80:
                result[intersection] = "Long Green"
            elif count >= 40:
                result[intersection] = "Normal Green"
            else:
                result[intersection] = "Short Green"

        return result

# Example
obj = TrafficControlSystem()
print(obj.add_intersection("5th Avenue & Main St", 85))
print(obj.add_intersection("Broadway & 1st St", 50))
print(obj.add_intersection("Central Square", 25))
print(obj.update_vehicle_count("Central Square", 30))
print(obj.get_congested_intersections(30))
print(obj.adjust_traffic_signals())
```

Easy Explanation

- `add_intersection`: stores intersection name and vehicle count.
- `update_vehicle_count`: updates an existing intersection; otherwise returns the required error message.
- `get_congested_intersections`: returns intersections where vehicle count is greater than the threshold.
- `adjust_traffic_signals`: >80 = Long Green, 40–80 = Normal Green, otherwise Short Green.

OOPS Exam Quick Revision

Dictionary update:

```
if key in self.data:  
    self.data[key] += value  
else:  
    self.data[key] = value
```

Dictionary subtraction:

```
self.data[key] -= value
```

Check key:

```
if key not in self.data:
```

Raise error:

```
raise ValueError("message")
```

Delete key:

```
del self.data[key]
```

Dictionary loop:

```
for key, value in self.data.items()
```

Filter:

```
[key for key, value in data.items() if value > 0]
```